Data 420 Final Project

William Makino

Business Understanding

A new ride-share platform intends to enter the NYC market, Empire. To compete against the big players (notably Uber and Lyft), it needs to understand demand patterns, base performance, and vehicle deployment strategies across time. The NYC TLC's FHV Base Aggregate Report provides monthly totals of dispatched trips, dispatched shared trips, and unique dispatched vehicles by base, which can be used to study market dynamics and estimate demand, most profitable segments, amount of vehicles needed for operation.

Business Questions

1. How many unique vehicles are required to meet a target share of monthly trips?

2. What natural groupings exist among NYC ride-share companies when considering their monthly trips and active vehicles?

3. Can we distinguish the big players (Uber/Lyft) from smaller ones using only aggregate data?

Data Understanding

Data source: https://data.cityofnewyork.us/Transportation/FHV-Base-Aggregate-Report/2v9c-2k7f/about_data

Loading Libraries

```r
library(tidyverse)
library(ggplot2)
library(janitor)
library(rpart)
library(rpart.plot)
```

Importing Data

```r
df_rideshare <- read_csv("NY_FHV.csv")
```

```
Rows: 52154 Columns: 8
-- Column specification --------------------------------------------------------
Delimiter: ","
chr (4): Base License Number, Base Name, DBA, Month Name
dbl (4): Year, Month, Total Dispatched Trips, Unique Dispatched Vehicles

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
summary(df_rideshare)
```

```
 Base License Number  Base Name             DBA                 Year      
 Length:52154        Length:52154       Length:52154       Min.   :2015  
 Class :character    Class :character   Class :character   1st Qu.:2017  
 Mode  :character    Mode  :character   Mode  :character   Median :2019  
                                                           Mean   :2019  
                                                           3rd Qu.:2021  
                                                           Max.   :2024  
     Month         Month Name        Total Dispatched Trips
 Min.   : 1.000   Length:52154       Min.   :       1      
 1st Qu.: 3.000   Class :character   1st Qu.:     304      
 Median : 6.000   Mode  :character   Median :    1227      
 Mean   : 6.484                      Mean   :   35068      
 3rd Qu.:10.000                      3rd Qu.:    3924      
 Max.   :12.000                      Max.   :17247995      
 Unique Dispatched Vehicles
 Min.   :    1.0           
 1st Qu.:    7.0           
 Median :   15.0           
```

```
 Mean   :  253.7
 3rd Qu.:   40.0
 Max.   :79178.0
```

```r
head(df_rideshare)
```

```
# A tibble: 6 x 8
  `Base License Number` `Base Name`                DBA    Year Month `Month Name`
  <chr>                 <chr>                      <chr> <dbl> <dbl> <chr>
1 B03324                PORT RICHMOND BLACK CAR ~ <NA>   2024     2 February
2 B02594                FENIX MOBILE DISPATCHERS~ <NA>   2022     3 March
3 B00112                AYY&M TRANSPORTATION, LLC HARB~  2015    10 October
4 B02401                NORTHSIDE BK LLC          <NA>   2021     7 July
5 B02868                CHA CHA TRANSPORTATION C~ <NA>   2019     3 March
6 B02832                NYC FLY WHEELS INC.       <NA>   2023     1 January
# i 2 more variables: `Total Dispatched Trips` <dbl>,
#   `Unique Dispatched Vehicles` <dbl>
```

Data Preparation

Using a new library I found that cleans feature names that are capitalized and contain spaces, formatting them to the standard lower case + underscores.

```r
df_rideshare <- df_rideshare %>%
  clean_names()
```

Merging Month and Year data into a Date column.

```r
df_rideshare <- df_rideshare %>%
  mutate(
    month = as.integer(month),
    year  = as.integer(year),
    date = paste0(year, "-", sprintf("%02d", month), "-01") %>% ymd()
  ) %>%
  relocate(date, .after = 3)
```

Checking NAs for the important features.

```r
df_rideshare %>%
  summarize(
    total = n(),
    na_trips    = sum(is.na(total_dispatched_trips)),
    na_vehicles = sum(is.na(unique_dispatched_vehicles)),
    na_name     = sum(is.na(base_name)),
    na_date     = sum(is.na(date))
  )
```

```
# A tibble: 1 x 5
  total na_trips na_vehicles na_name na_date
  <int>    <int>       <int>   <int>   <int>
1 52154        0           0       0       0
```

New Feature - Quarter

```r
df_rideshare <- df_rideshare %>%
  mutate(
    quarter = quarter(date)
    ) %>%
  relocate(quarter, .after = 5)
```

New Feature - Trips per Vehicle

```r
df_rideshare <- df_rideshare %>%
  mutate(
    trips_per_vehicle = total_dispatched_trips / unique_dispatched_vehicles
    )
```

New Feature - Total Trips with a lag of 1

```r
df_rideshare <- df_rideshare %>%
  arrange(base_license_number, date) %>%
  group_by(base_license_number) %>%
  mutate(
    trips_lag_1 = dplyr::lag(total_dispatched_trips, 1)
  ) %>%
  ungroup()
```

New Feature - Moving Averages of 3 and 6

```r
df_rideshare <- df_rideshare %>%
  arrange(base_license_number, date) %>%
  group_by(base_license_number) %>%
  mutate(
    trips_lag_1 = dplyr::lag(total_dispatched_trips, 1),
    trips_lag_2 = dplyr::lag(total_dispatched_trips, 2)
  ) %>%
  ungroup() %>%
  rowwise() %>%
  mutate(
    trips_ma_3 = mean(c(total_dispatched_trips, trips_lag_1, trips_lag_2), na.rm = FALSE)
  ) %>%
  ungroup()
```

```r
df_rideshare <- df_rideshare %>%
  arrange(base_license_number, date) %>%
  group_by(base_license_number) %>%
  mutate(
    trips_lag_3 = dplyr::lag(total_dispatched_trips, 3),
    trips_lag_4 = dplyr::lag(total_dispatched_trips, 4),
    trips_lag_5 = dplyr::lag(total_dispatched_trips, 5)
  ) %>%
  ungroup() %>%
  rowwise() %>%
  mutate(
    trips_ma_6 = mean(
      c(total_dispatched_trips,
        trips_lag_1, trips_lag_2, trips_lag_3, trips_lag_4, trips_lag_5),
      na.rm = FALSE
    )
  ) %>%
  ungroup() %>%
  select(-trips_lag_1, -trips_lag_2, -trips_lag_3, -trips_lag_4, -trips_lag_5)
```

Creating a normalized version of the data.

```r
df_normalized <- df_rideshare %>%
  mutate(across(c(total_dispatched_trips, unique_dispatched_vehicles, trips_per_vehicle), sca
```

Split the data between train and test (for both df_rideshare and df_normalized), leaving 2015-2022 for training and 2023-2024 for testing.

```r
cutoff_year <- 2023

df_rideshare_train <- df_rideshare %>%
  filter(year < cutoff_year)

df_rideshare_test <- df_rideshare %>%
  filter(year >= cutoff_year)

df_normalized_train <- df_normalized %>%
  filter(year < cutoff_year)

df_normalized_test <- df_normalized %>%
  filter(year >= cutoff_year)
```

Modeling & Evaluation

Multiple Linear Regression

Prepare month as a factor so it treats seasonality as 12 separate dummy variables so the model can learn different average trip levels for each month instead of assuming a straight-line trend over months. This captures seasonal shifts like winter dips or summer peaks.

```r
df_rideshare_lm <- df_rideshare %>%
  mutate(month_f = factor(month, levels = 1:12, labels = month.abb))
```

Fit the multiple linear regression with total_dispatched_trips as response and unique_dispatched_vehicles + month_f as predictors.

```r
mod_trips <- lm(
  total_dispatched_trips ~ unique_dispatched_vehicles + month_f,
  data = df_rideshare_lm
)

summary(mod_trips)
```

```
Call:
lm(formula = total_dispatched_trips ~ unique_dispatched_vehicles +
    month_f, data = df_rideshare_lm)
```

```
Residuals:
     Min       1Q   Median       3Q      Max 
-4623406     2928     4576     5691  4793038 

Coefficients:
                            Estimate Std. Error t value Pr(>|t|)    
(Intercept)                -5566.3487  2490.2498  -2.235   0.0254 *  
unique_dispatched_vehicles   159.1545     0.2219 717.301   <2e-16 ***
month_fFeb                  -218.8813  3541.8757  -0.062   0.9507    
month_fMar                  1613.3779  3526.5265   0.457   0.6473    
month_fApr                   283.2517  3513.5659   0.081   0.9357    
month_fMay                  1095.9708  3579.9434   0.306   0.7595    
month_fJun                   308.0744  3564.2976   0.086   0.9311    
month_fJul                    33.7308  3580.8235   0.009   0.9925    
month_fAug                  -698.4744  3565.3661  -0.196   0.8447    
month_fSep                  -843.4734  3579.4980  -0.236   0.8137    
month_fOct                   636.8422  3532.9330   0.180   0.8570    
month_fNov                  -607.3515  3544.1359  -0.171   0.8639    
month_fDec                  1386.6872  3518.6311   0.394   0.6935    
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 166500 on 52141 degrees of freedom
Multiple R-squared:  0.908, Adjusted R-squared:  0.908 
F-statistic: 4.288e+04 on 12 and 52141 DF,  p-value: < 2.2e-16
```

- About 90.8% of the variation in monthly total_dispatched_trips is explained by the predictors.
- Given the p-values, unique_dispatched_vehicles is by far the most significant in this model, with the monthly predictors all falling over the level of 0.05.
- The vehicles coefficient is the star: on average, adding one active vehicle is associated with ~160 more trips per month.

```r
last_period <- df_rideshare %>%
  summarize(last_date = max(date)) %>%
  pull(last_date)

# 2) Filter to that period and brand-match Uber/Lyft in base name
#    Common patterns include "UBER", "UBER TECHNOLOGIES", "LYFT", "LYFT, INC.", etc.
brand_totals <- df_rideshare %>%
  filter(date == last_period) %>%
```

```r
  mutate(
    brand = case_when(
      str_detect(toupper(base_license_number), "UBER") ~ "Uber",
      str_detect(toupper(base_license_number), "LYFT") ~ "Lyft",
      TRUE ~ NA_character_
    )
  ) %>%
  filter(!is.na(brand)) %>%
  summarize(
    last_period = first(date),
    total_dispatched_trips = sum(total_dispatched_trips, na.rm = TRUE),
    .by = brand
  ) %>%
  arrange(desc(total_dispatched_trips))

brand_totals
```

```
# A tibble: 2 x 3
  brand last_period total_dispatched_trips
  <chr> <date>                       <dbl>
1 Uber  2024-04-01                14704197
2 Lyft  2024-04-01                 5028841
```

Considering Uber had 14,704,197 and Lyft had 5,028,841 total trips in the last recorded month (April/24), and the linear model indicates that for every vehicle in the "fleet" equals to approximately 160 trips, to match the big players numbers Empire would need to have ~31,597 cars to match Lyft and ~92,389 to match Uber.

```r
brand_vehicles <- df_rideshare %>%
  filter(date == last_period) %>%
  mutate(
    brand = case_when(
      str_detect(toupper(base_license_number), "UBER") ~ "Uber",
      str_detect(toupper(base_license_number), "LYFT") ~ "Lyft",
      TRUE ~ NA_character_
    )
  ) %>%
  filter(!is.na(brand)) %>%
  summarize(
    last_period = first(date),
    unique_dispatched_vehicles = sum(unique_dispatched_vehicles, na.rm = TRUE),
```

```r
    .by = brand
  ) %>%
  arrange(desc(unique_dispatched_vehicles))

brand_vehicles
```

```
# A tibble: 2 x 3
  brand last_period unique_dispatched_vehicles
  <chr> <date>                           <dbl>
1 Uber  2024-04-01                       75353
2 Lyft  2024-04-01                       59662
```

```r
brand_month <- df_rideshare %>%
  filter(date == last_period) %>%
  mutate(
    brand = case_when(
      str_detect(toupper(base_license_number), "UBER") ~ "Uber",
      str_detect(toupper(base_license_number), "LYFT") ~ "Lyft",
      TRUE ~ NA_character_
    )
  ) %>%
  filter(!is.na(brand)) %>%
  summarize(
    trips = sum(total_dispatched_trips, na.rm = TRUE),
    vehicles = sum(unique_dispatched_vehicles, na.rm = TRUE),
    tpv_realized = trips / vehicles,
    .by = brand
  )

brand_month
```

```
# A tibble: 2 x 4
  brand    trips vehicles tpv_realized
  <chr>    <dbl>    <dbl>        <dbl>
1 Lyft   5028841    59662         84.3
2 Uber  14704197    75353        195.
```

In reality, the culture of each company with it's drivers changes what the number of needed vehicles really is. For example, Uber apparently pushes for unique vehicles to make more trips (more than double) than Lyft does. Or maybe this is the result of Uber being the first choice for drivers, and Lyft being used as an alternative.

Whatever the reason is, this makes Uber reach that number of trips with way less unique cars than Lyft does. So in reality, the number of needed cars on the fleet would be in-between these two numbers, or even higher than Lyft, considering it'd be a new player with less market power. Maybe doing promotional events and prices could catch the attention of drivers and riders for an accelerated inauguration.

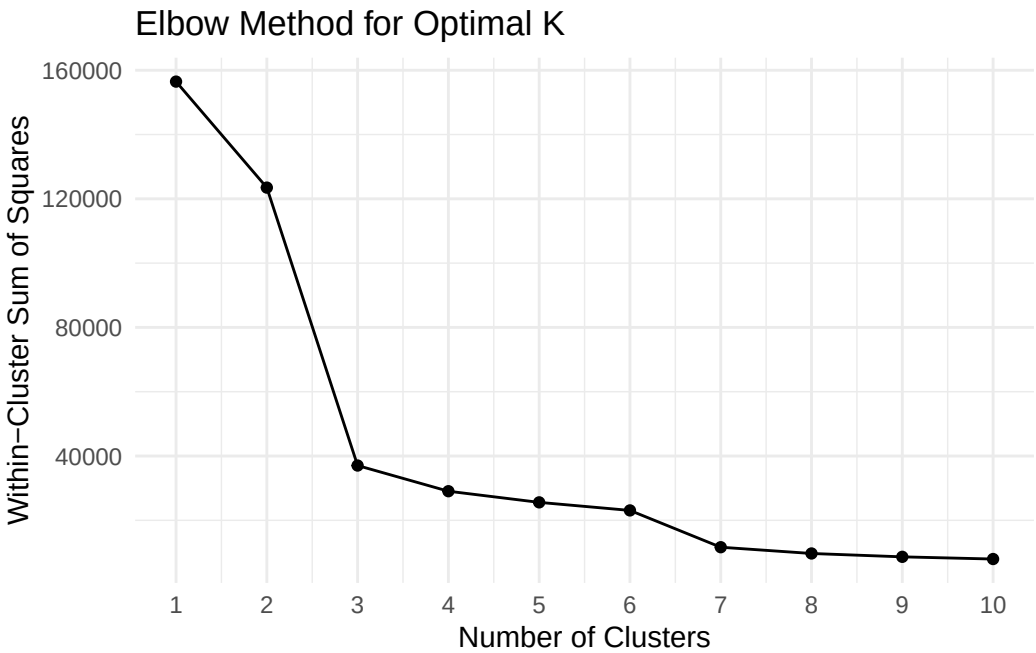
K-Means

Start by doing the elbow test to find the best k-value.

```
set.seed(1)

# Compute WCSS for k = 1 to 10
wss <- map_dbl(1:10, function(k) {
  kmeans(df_normalized[9:11], centers = k, nstart = 25, algorithm="MacQueen", iter.max = 100)
})
```

```
Warning: did not converge in 100 iterations
```

```
# Visualize the Elbow Plot
tibble(k = 1:10, wss = wss) %>%
  ggplot(aes(x = k, y = wss)) +
  geom_line() +
  geom_point() +
  scale_x_continuous(breaks = 1:10) +
  labs(title = "Elbow Method for Optimal K",
       x = "Number of Clusters", y = "Within-Cluster Sum of Squares") +
  theme_minimal()
```



Running k-means with a k-value of 2 with the normalized data frame. Prints out how many entries were allocated in each cluster.

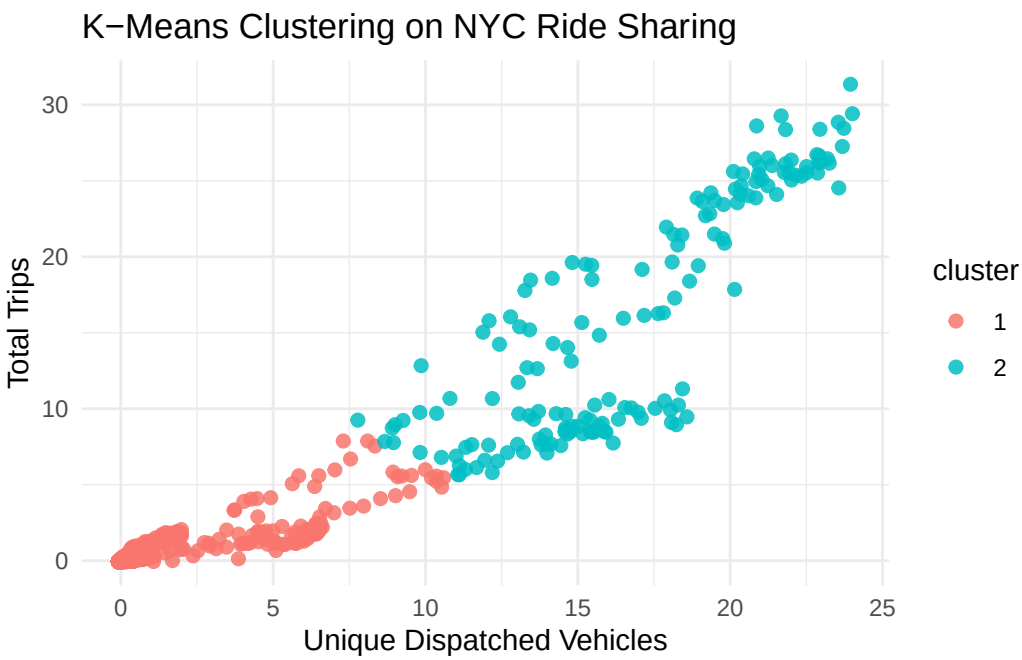
```
set.seed(1)
k <- 2

km_res <- kmeans(df_normalized %>%
                   select(total_dispatched_trips, unique_dispatched_vehicles, trips_per_vehi

df_normalized <- df_normalized %>%
  mutate(cluster = factor(km_res$cluster))

table(km_res$cluster)
```

```
    1     2
51990   164
```

Visualizing the clusters on a scatter plot.

```
ggplot(df_normalized, aes(x = unique_dispatched_vehicles, y = total_dispatched_trips, color =
  geom_point(size = 2, alpha = 0.85) +
```

```
  labs(title = "K-Means Clustering on NYC Ride Sharing",
       x = "Unique Dispatched Vehicles",
       y = "Total Trips") +
  theme_minimal()
```



Checking in which clusters the Uber and Lyft records fell in.

```
df_normalized <- df_normalized %>%
  mutate(
    brand = case_when(
      str_detect(base_license_number, "UBER") ~ "Uber",
      str_detect(base_license_number, "LYFT") ~ "Lyft",
      TRUE ~ "Other"
    )
  )

table_brand_cluster <- df_normalized %>%
  count(brand, cluster) %>%
  arrange(brand, cluster)

table_brand_cluster
```

```
# A tibble: 5 x 3
```

```
  brand cluster     n
  <chr> <fct>   <int>
1 Lyft  1          44
2 Lyft  2          65
3 Other 1       51933
4 Uber  1          13
5 Uber  2          99
```

In this instance, considering each dot is an aggregate month for each company, the model split the data into 1- low total trips + low unique vehicles and 2- high total trips + high unique vehicles. Uber tends to fall into cluster 2 more often than Lyft does, probably because it's ride volume is just bigger, having more total trips and unique vehicles on average. What is hidden in this visual is the huge amount of entries overlapped on the very low end of each axis, representing the small companies that have such a smaller scale compared to the big players.

Decision Tree

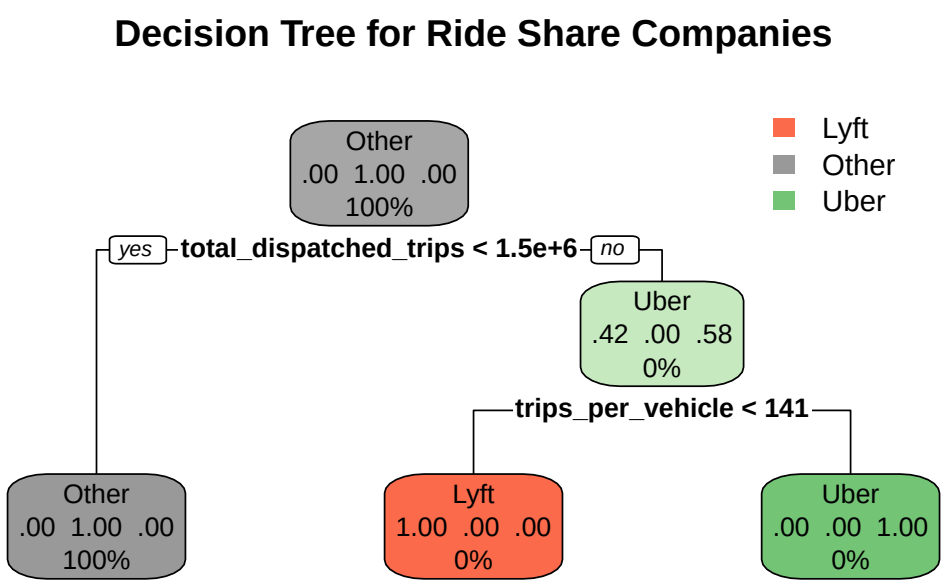
Preparation to do a decision tree that classifies the records between Uber, Lyft and Others.

```r
train_classify <- df_rideshare_train %>%
  mutate(
    brand = case_when(
      str_detect(toupper(base_license_number), "UBER") ~ "Uber",
      str_detect(toupper(base_license_number), "LYFT") ~ "Lyft",
      TRUE ~ "Other"
    )
  ) %>%
  select(brand,
         unique_dispatched_vehicles,
         trips_per_vehicle,
         total_dispatched_trips) %>%
  mutate(brand = factor(brand))

test_classify <- df_rideshare_test %>%
  mutate(
    brand = case_when(
      str_detect(toupper(base_license_number), "UBER") ~ "Uber",
      str_detect(toupper(base_license_number), "LYFT") ~ "Lyft",
      TRUE ~ "Other"
    )
  ) %>%
```

```
  select(brand,
         unique_dispatched_vehicles,
         trips_per_vehicle,
         total_dispatched_trips) %>%
  mutate(brand = factor(brand, levels = levels(train_classify$brand)))
```

Running the classifier Decision Tree.

```
set.seed(1)

tree_classify <- rpart(
  brand ~ unique_dispatched_vehicles + trips_per_vehicle + total_dispatched_trips,
  data = train_classify,
  method = "class",
  control = rpart.control(minsplit = 150)
)

rpart.plot(tree_classify, main = 'Decision Tree for Ride Share Companies')
```




```
printcp(tree_classify)
```

```
Classification tree:
rpart(formula = brand ~ unique_dispatched_vehicles + trips_per_vehicle +
    total_dispatched_trips, data = train_classify, method = "class",
    control = rpart.control(minsplit = 150))

Variables actually used in tree construction:
[1] total_dispatched_trips trips_per_vehicle

Root node error: 189/44940 = 0.0042056

n= 44940

       CP nsplit rel error  xerror     xstd
1 0.50794      0   1.00000 1.00000 0.072586
2 0.37037      1   0.49206 0.50794 0.051786
3 0.01000      2   0.12169 0.44444 0.048448
```

```r
pred_lab_classify <- predict(tree_classify, newdata = test_classify, type = "class")
acc_classify <- mean(pred_lab_classify == test_classify$brand)
conf_classify <- table(Predicted = pred_lab_classify, Actual = test_classify$brand)

acc_classify
```

```
[1] 1
```

```r
conf_classify
```

```
         Actual
Predicted Lyft Other Uber
    Lyft    16     0    0
    Other    0  7182    0
    Uber     0     0   16
```

Apparently with the given data, it is really easy to tell apart Uber, Lyft and Others using Decision Trees. It used concepts that were detected previously in this analysis: low number of total trips has to be Others, since the big players have a bigger volume of trips, then to split between the two dominant it's just a case of looking at trips per vehicle, which has came previously that Uber tends to have much more trips per unique vehicle in average, making it very easy to split them apart. The magic numbers are: Less than 1.5M total trips = Others More or equal to 1.5M total trips = Uber or Lyft Less than 141 trips per vehicle: Lyft More or equal to 141 trips per vehicle: Uber

Summary & Conclusion

Overall, all the methods worked very well.
Linear regression showed a strong correlation between total trips and unique vehicles and showed that for each new vehicle in the fleet, there is an increase of ~160 new trips per month.
K-Means had a good result, but a very predictable and logic one. The clustering it came up with was splitting the data in 2: low and high volume.
Decision Tree also ended up having an easy task and 100% correctly splitting the monthly aggregates into Uber, Lyft and Others.